

Machine Learning — Detailed Notes (Unit 1 & Unit 2)

UE24CS352A — ISA 1 Preparation

Pattern: 16×1 mark + 4×2 marks + 4×4 marks = 40 marks

UNIT 1: Introduction, Performance Metrics, Classification with Decision Trees and KNN

1. Introduction to Machine Learning

1.1 What is Machine Learning?

Machine Learning is a field of study that gives computers the ability to learn from data without being explicitly programmed. Instead of hard-coding rules, an ML system **learns a function/pattern from examples (data)** and uses it to make predictions on new, unseen data.

Formal Definition (Tom Mitchell, 1997):

A computer program is said to **learn** from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in **T**, as measured by **P**, improves with experience **E**.

A well-defined learning problem is specified as $\langle \mathbf{P}, \mathbf{T}, \mathbf{E} \rangle$:

- **T (Task):** What the program is being asked to do (e.g., classify email as spam/not spam)
- **P (Performance measure):** How success is measured (e.g., accuracy)
- **E (Experience):** The data/experience used to learn (e.g., a set of labeled emails)

PTE Examples (classic, exam-favorite):

Problem	Task (T)	Performance (P)	Experience (E)
Checkers	Playing checkers	% of games won	Games played against itself
Handwriting Recognition	Classify handwritten words from images	% of words correctly classified	Database of labeled handwritten words
Self-driving car	Steer a vehicle using vision sensors	Avg. distance traveled before error	Sequence of images + steering commands from human driver
Text Categorization	Classify documents into categories	% of documents correctly classified	Labeled document corpus
Spam Detection	Classify email as spam/not spam	% of emails correctly classified	Labeled emails (spam/not spam) marked by users
Credit Card Fraud	Classify transaction as fraud/not fraud	% correctly flagged, false alarms	Historical labeled transactions

1.2 ML as a Flow / 7 Steps of Machine Learning

1. **Data Collection** — gather relevant raw data
2. **Data Preparation** — clean, format, handle missing values, normalize
3. **Choosing a Model** — pick algorithm suited to the problem (regression, classification, clustering, etc.)
4. **Training** — fit the model on training data (learn parameters)
5. **Evaluation** — test on unseen (validation/test) data
6. **Parameter Tuning** — adjust hyperparameters to improve performance
7. **Prediction** — deploy model to make predictions on new data

1.3 Types of Machine Learning Models

- **Supervised Learning:** Learn a mapping from input X to output Y using labeled data.
 - **Classification** — output is discrete/categorical (e.g., spam/not spam)
 - **Regression** — output is continuous (e.g., predicting house price)
- **Unsupervised Learning:** Learn structure/patterns from unlabeled data (e.g., clustering, dimensionality reduction)
- **Reinforcement Learning:** An agent learns by interacting with an environment, receiving rewards/penalties, to maximize cumulative reward.
- **Semi-supervised Learning:** Uses a small amount of labeled data with a large amount of unlabeled data.

2. Concept Learning

2.1 Concept Learning — Definition

Concept Learning is the task of inferring a boolean-valued function (a **concept**) from training examples, i.e., learning which examples are positive (belong to the concept) and which are negative, by searching through a hypothesis space.

Formally: Given a set of training examples of a target concept c , find a hypothesis h such that $h(x) = c(x)$ for all x .

2.2 Hypothesis

- A **hypothesis** (h) is a candidate function/rule that approximates the target concept c . It maps instances (inputs) to $\{0, 1\}$ — i.e., member/non-member of the concept.
- Hypotheses are typically expressed as a conjunction of constraints on the instance attributes. Each attribute constraint can be:
 - A specific value (e.g., $\text{Sky} = \text{Sunny}$)
 - $?$ — any value is acceptable (don't care)
 - $\emptyset$ — no value is acceptable (this hypothesis rejects all instances)

Example (EnjoySport concept): $h = \langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same} \rangle$ means: Sky must be Sunny, Temp must be Warm, Humidity can be anything, Wind must be Strong, Water must be Warm, Forecast must be Same.

2.3 More-General-Than / More-Specific-Than Relation

Hypothesis h_1 is **more general than or equal to** h_2 (written $h_1 \geq_g h_2$) if every instance satisfying h_2 also satisfies h_1 . This defines a **partial order** over the hypothesis space, which enables efficient search.

2.4 Version Space

The **Version Space** (VS) with respect to hypothesis space H and training examples D is the subset of hypotheses in H that are **consistent** with all training examples in D .

$$VS_{H,D} = \{ h \in H \mid \text{Consistent}(h, D) \}$$

A hypothesis is *consistent* with a training example if it correctly classifies that example.

Candidate-Elimination Algorithm

Represents the version space compactly using two boundary sets:

- **General boundary** (G): the set of maximally general hypotheses consistent with D
- **Specific boundary** (S): the set of maximally specific hypotheses consistent with D

Algorithm:

1. Initialize G to the set of maximally general hypotheses (i.e., $\langle ?, ?, \dots, ? \rangle$)

2. Initialize S to the set of maximally specific hypotheses (i.e., $\langle \emptyset, \emptyset, \dots, \emptyset \rangle$)
3. For each training example d :
 - **If d is a positive example:**
 - Remove from G any hypothesis inconsistent with d
 - For each hypothesis s in S inconsistent with d : remove s , add all minimal generalizations of s consistent with d (and having some more general hypothesis in G)
 - Remove from S any hypothesis more general than another in S
 - **If d is a negative example:**
 - Remove from S any hypothesis inconsistent with d
 - For each hypothesis g in G inconsistent with d : remove g , add all minimal specializations of g consistent with d (and having some more specific hypothesis in S)
 - Remove from G any hypothesis less general than another in G
4. The final version space is bounded by S and G — every hypothesis between them (more general than some $s \in S$ and more specific than some $g \in G$) is in the version space.

2.5 Inductive Bias

Since a learner can only see a finite set of training examples, it must make assumptions beyond the data to generalize to unseen instances — this is called **inductive bias**.

Inductive Bias (formal): The set of assumptions that, together with the training data, deductively justify the classifications the learner assigns to future instances.

Examples of inductive bias:

- **Candidate-Elimination / FIND-S:** The target concept is contained in the hypothesis space H (restriction bias).
- **Decision Tree (ID3):** Prefers shorter trees, and trees with high-information-gain attributes near the root (preference bias / Occam's razor).
- **k-NN:** Assumes that instances close to each other (by some distance metric) have similar output values.

A learner with **no inductive bias** cannot generalize beyond the observed training data (it can only classify examples it has already seen).

3. Performance Metrics

3.1 Confusion Matrix

For binary classification (Positive/Negative classes):

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

3.2 Accuracy

Proportion of total predictions that are correct.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Misleading on **imbalanced datasets** (e.g., 95% negative class → predicting all negative gives 95% accuracy but is useless).

3.3 Precision

Of all instances predicted positive, how many are actually positive. Measures **exactness**.

$$\text{Precision} = TP / (TP + FP)$$

Important when the **cost of False Positives is high** (e.g., spam detection — don't want to mark genuine mail as spam).

3.4 Recall (Sensitivity / True Positive Rate / Hit Rate)

Of all actual positive instances, how many were correctly identified. Measures **completeness**.

$$\text{Recall} = \text{Sensitivity} = \text{TPR} = TP / (TP + FN)$$

Important when the **cost of False Negatives is high** (e.g., disease detection — don't want to miss a sick patient).

3.5 Specificity (True Negative Rate)

Of all actual negative instances, how many were correctly identified as negative.

$$\text{Specificity} = \text{TNR} = TN / (TN + FP)$$

3.6 False Positive Rate (FPR)

$$\text{FPR} = 1 - \text{Specificity} = FP / (TN + FP)$$

3.7 F1-Score

Harmonic mean of Precision and Recall — useful when you need a balance between the two.

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

3.8 ROC Curve (Receiver Operating Characteristics)

- A **probability curve** that plots **TPR (Sensitivity) on Y-axis** against **FPR (1 – Specificity) on X-axis** at various classification threshold settings.
- Shows the tradeoff between benefits (TP) and costs (FP) as the decision threshold changes.
- **Point (0,1)** = perfect classification (100% TPR, 0% FPR)
- **Diagonal line (y = x)** = random guessing
- A point is "better" if it is more to the **top-left** — higher TPR, lower FPR.

3.9 AUC (Area Under the ROC Curve)

- Represents the degree of separability — how well the model distinguishes between classes.
- $AUC = 1 \rightarrow$ perfect classifier
- $AUC = 0.5 \rightarrow$ no discriminative power (random guessing)
- Higher AUC = better model. Used to **compare multiple classifiers** irrespective of threshold.

4. Bias-Variance Decomposition and Tradeoff

4.1 Sources of Error

The expected prediction error of a model can be decomposed as:

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error (noise)}$$

4.2 Bias

- **Bias** is the error due to overly simplistic assumptions in the learning algorithm.
- High bias $\rightarrow$ model **underfits** — fails to capture the underlying pattern (e.g., fitting a straight line to non-linear data).
- Low bias = model is flexible/complex enough to fit training data well.

4.3 Variance

- **Variance** is the error due to the model's sensitivity to small fluctuations in the training data.

- High variance → model **overfits** — fits noise in training data, performs poorly on new/unseen data.
- Low variance = model output is stable even if training data changes slightly.

4.4 Bias-Variance Tradeoff

- Simple models (e.g., linear regression, shallow decision trees) → **High Bias, Low Variance**
- Complex models (e.g., deep decision trees, high-degree polynomial regression) → **Low Bias, High Variance**
- Goal: find the sweet spot (optimal model complexity) that minimizes total error = $\text{Bias}^2 + \text{Variance}$.

Model complexity	Bias	Variance	Behavior
Too simple	High	Low	Underfitting
Too complex	Low	High	Overfitting
Optimal	Balanced	Balanced	Good generalization

4.5 Cross-Validation Strategies

Used to estimate model performance on unseen data and to tune hyperparameters, reducing dependency on a single train/test split.

- **Hold-out Validation:** Split data once into train and test (e.g., 70/30 or 80/20).
- **k-Fold Cross-Validation:** Split data into k equal folds. Train on $k-1$ folds, test on the remaining fold; repeat k times (each fold used once as test set). Average the results.
- **Leave-One-Out Cross-Validation (LOOCV):** Special case of k-fold where $k = n$ (number of samples) — each sample is used once as the test set. Computationally expensive but low bias.
- **Stratified k-Fold:** Like k-fold, but preserves the class proportion in each fold — important for imbalanced datasets.

5. Decision Trees

5.1 What is a Decision Tree?

A tree-structured classifier where:

- **Internal nodes** = test on an attribute
- **Branches** = outcome of the test
- **Leaf nodes** = class label (decision)

Decision trees classify instances by sorting them down the tree from root to leaf, testing one attribute at a time.

5.2 ID3 Algorithm (Basic Algorithm for Classification)

ID3 (Iterative Dichotomiser 3) builds a decision tree top-down, greedily selecting at each step the attribute that best splits the data, using **Information Gain** as the selection heuristic (unlike CART, which uses **Gini Impurity**).

ID3 Algorithm steps:

1. If all examples belong to the same class → return a leaf node with that class.
2. If no attributes remain → return a leaf with the majority class.
3. Otherwise, select the attribute $\boxed{A}$ with the **highest Information Gain**.
4. Create a decision node for $\boxed{A}$; for each possible value of $\boxed{A}$, create a branch and recursively build the subtree using the subset of examples with that value.

5.3 Entropy

Entropy measures the **impurity/uncertainty/non-homogeneity** of a set of examples. Entropy needs to be reduced for a "pure" classification.

$$\text{Entropy}(S) = H(S) = -\sum p(x_i) \log_2(p(x_i)), \quad i = 1 \text{ to } c$$

where $\boxed{p(x_i)}$ is the proportion of examples in S belonging to class $\boxed{i}$, and $\boxed{c}$ is the number of classes.

- Entropy = 0 → all examples belong to one class (pure)
- Entropy = 1 (for binary) → examples are evenly split (maximum impurity)

Example: For a set S with 9 positive and 5 negative examples ($|S| = 14$):

$$\text{Entropy}(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.94$$

5.4 Information Gain

$$\text{Information Gain}(S, A) = \text{Entropy}(S) - \sum (|S_v|/|S|) \times \text{Entropy}(S_v)$$

i.e., **Information Gain** = **Entropy before split** – **Weighted Entropy after split**, where the sum is over each value $\boxed{v}$ of attribute $\boxed{A}$, and $\boxed{S_v}$ is the subset of S with $\boxed{A = v}$.

The attribute with the **highest Information Gain** is chosen as the splitting attribute at each node.

Worked Example (Outlook attribute, $|S| = 14$, Entropy(S) = 0.94):

$$\text{Gain}(S, \text{Outlook}) = \text{Entropy}(S) - I(\text{Outlook}) = 0.94 - 0.693 = 0.247$$

(Similarly compute Gain for Temperature, Humidity, Wind — pick the attribute with max gain as root.)

5.5 Decision Boundary for Decision Trees

- Decision trees create **axis-parallel (rectangular) decision boundaries** — since each split tests a single feature against a threshold, the boundary lines in feature space (x-y axis) are always **horizontal or vertical**, forming rectangular regions.
- This is unlike linear classifiers (which produce a single straight-line boundary at any angle), or KNN (which produces irregular boundaries).

5.6 Hypothesis Space Search and Inductive Bias of ID3

- ID3 searches a **complete hypothesis space** (space of all possible decision trees) — the target function is guaranteed to be in there.
- It performs **hill-climbing search**, maintaining only a single current hypothesis (no backtracking) — this differs from Candidate-Elimination, which maintains all consistent hypotheses.
- **Inductive Bias of ID3 (preference bias):** Shorter trees are preferred over longer ones; trees that place attributes with higher information gain closer to the root are preferred. (This is a *preference* bias, not a *restriction* bias — unlike Candidate-Elimination, ID3's hypothesis space is not restricted, but its search strategy prefers certain hypotheses.)

5.7 Issues in Decision Tree Learning

5.7.1 Overfitting

A tree overfits when it perfectly fits the training data (including noise) but performs poorly on unseen data. Formally, hypothesis (h) overfits the training data if there exists an alternative hypothesis (h') such that (h) has smaller error than (h') on training data, but (h') has smaller error than (h) over the entire distribution of instances.

Causes: Noisy training data, too few training examples, a tree grown too deep/complex.

5.7.2 Solutions to Overfitting

- **Pre-pruning (Early stopping):** Stop growing the tree early — e.g., stop if further splitting doesn't improve information gain beyond a threshold, or limit max depth / min samples per leaf.
- **Post-pruning:** Grow the full tree first, then prune back nodes that don't improve validation-set accuracy (e.g., **Reduced Error Pruning**, **Rule Post-Pruning** — convert tree to rules, then prune each rule independently).
- Use a **separate validation set** to decide when to stop growing / which nodes to prune.
- Use **Minimum Description Length (MDL)** — prefer smaller trees.

5.7.3 Dealing with Continuous-Valued Attributes

ID3 originally handles only discrete attributes. For continuous attributes:

- Dynamically define a **new discrete-valued (boolean) attribute** by choosing a threshold c that splits the values into two ranges ($A < c$ and $A \geq c$).
- To choose the threshold: sort examples by the attribute value, and consider candidate thresholds at the midpoint between consecutive examples where the class label changes. Pick the threshold that yields the maximum information gain.

5.8 Logistic Regression

- Despite the name, Logistic Regression is a **classification** algorithm (typically binary classification).
- It models the probability that an instance belongs to a particular class using the **sigmoid (logistic) function**:

$$P(y=1|x) = \sigma(z) = 1 / (1 + e^{(-z)}), \quad \text{where } z = w^T x + b$$

- Output is a probability between 0 and 1; a threshold (commonly 0.5) is applied to assign the final class label.
- **Decision boundary:** Linear in the feature space (a straight line/hyperplane), since $z = w^T x + b = 0$ defines the boundary.
- **Training:** Parameters (w, b) are learned by maximizing the likelihood of the observed data (equivalently, minimizing the **cross-entropy / log-loss** cost function) — typically via Gradient Descent.

$$\text{Cost} = -(1/N) \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

6. Instance-Based Learning: k-Nearest Neighbor (k-NN)

6.1 Concept

k-NN is a **lazy learning / instance-based** algorithm — it does not build an explicit model during training; instead, it stores all training instances and defers computation until a query (prediction) is made.

Algorithm:

1. Choose the number of neighbors k .
2. Compute the distance (e.g., Euclidean) between the query instance and all training instances.
3. Select the k nearest training instances.
4. **For Classification:** assign the majority class among the k neighbors (majority voting).
5. **For Regression:** assign the average (mean) of the target values of the k neighbors.

6.2 Distance Metrics

Euclidean Distance: $d(x, y) = \sqrt{\sum (x_i - y_i)^2}$

Manhattan Distance: $d(x, y) = \sum |x_i - y_i|$

6.3 k-NN for Classification vs Regression

- **Classification:** Discrete output — majority vote among k nearest neighbors' labels.
- **Regression:** Continuous output — average (or distance-weighted average) of the k nearest neighbors' target values.

6.4 Choice of k

- **Small k** (e.g., k=1) → sensitive to noise, low bias, high variance → can overfit.
- **Large k** → smoother decision boundary, more robust to noise, but may underfit (high bias) and blur class boundaries.
- k is usually chosen via cross-validation. Often an **odd k** is used for binary classification to avoid ties.

6.5 Decision Boundary for k-NN

- k-NN produces a **highly irregular, non-linear decision boundary** that adapts closely to the local distribution of the training data (unlike the axis-parallel boundary of decision trees or the linear boundary of logistic regression).
- For k=1, the boundary is formed by a **Voronoi diagram** — each region belongs to the class of its nearest training point.
- As k increases, the boundary becomes smoother.

6.6 Key Characteristics

- **Inductive bias of k-NN:** Instances that are close together in feature space are assumed to have similar target values ("smoothness assumption").
 - No explicit training phase (all computation deferred to prediction time) → prediction can be slow for large datasets ("lazy learner").
 - Sensitive to **feature scaling** — features must typically be normalized/standardized since distance metrics are scale-dependent.
 - Suffers from the **curse of dimensionality** — distance metrics become less meaningful in high-dimensional space.
-

UNIT 2: Artificial Neural Networks (ANN) and Support Vector Machines (SVM)

7. Artificial Neural Networks — Introduction

7.1 Motivation

ANNs are inspired by the structure and function of biological neural networks (the brain). They consist of interconnected units ("neurons") organized in layers, capable of learning complex, non-linear relationships between inputs and outputs.

7.2 Perceptron

The **Perceptron** is the simplest type of artificial neuron, and the basic building block of ANNs. Introduced by Frank Rosenblatt (1958).

Structure:

- Takes a vector of real-valued inputs $(x_1, x_2, \dots, x_n)$
- Computes a weighted sum plus a bias: $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w \cdot x + b$
- Passes z through an **activation function** to produce the output.

Perceptron output (using step function):

$$o(x) = 1 \quad \text{if } w \cdot x + b > 0$$
$$o(x) = -1 \text{ (or } 0) \quad \text{otherwise}$$

Perceptron Learning Rule:

$$w_i \leftarrow w_i + \Delta w_i, \quad \text{where } \Delta w_i = \eta(t - o)x_i$$

- η = learning rate, t = target output, o = perceptron's actual output.
- Weights are updated only when the perceptron misclassifies an example.
- **Limitation:** A single perceptron can only learn **linearly separable** functions (e.g., it cannot represent XOR).

7.3 Multi-Layer Networks and Backpropagation

7.3.1 Multi-Layer Perceptron (MLP)

To overcome the linear-separability limitation of a single perceptron, **Multi-Layer Networks** stack multiple layers of neurons:

- **Input layer** — receives the feature vector
- **Hidden layer(s)** — perform intermediate non-linear transformations
- **Output layer** — produces the final prediction

Each neuron in a layer is (typically fully) connected to every neuron in the next layer, each connection having a weight. With non-linear activation functions, MLPs can approximate any continuous function (**Universal Approximation Theorem**) and represent non-linear decision boundaries (e.g., XOR).

7.3.2 Backpropagation Algorithm

Backpropagation is the algorithm used to **train** multi-layer networks by efficiently computing the gradient of the loss function with respect to every weight in the network, using the **chain rule** of calculus.

Steps:

1. **Forward Pass:** Input is passed through the network layer by layer; each neuron computes a weighted sum and applies an activation function; final output is produced at the output layer.
2. **Compute Loss:** Compare predicted output to the true target using a loss function (e.g., Mean Squared Error, Cross-Entropy).
3. **Backward Pass:** Propagate the error backward from the output layer to the input layer, computing the gradient of the loss with respect to each weight using the chain rule:

$$\partial E / \partial w_{ij} = \partial E / \partial o_j \times \partial o_j / \partial \text{net}_j \times \partial \text{net}_j / \partial w_{ij}$$

4. **Weight Update:** Update each weight in the direction that reduces the error (gradient descent):

$$w_{ij} \leftarrow w_{ij} - \eta (\partial E / \partial w_{ij})$$

5. Repeat over many epochs until the error converges to an acceptable minimum.

Key idea: Error at the output layer is propagated backward to compute how much each hidden-layer weight contributed to the error (using the derivative of the activation function).

7.4 Activation Functions

Activation functions introduce **non-linearity**, enabling the network to learn complex patterns (without them, a multi-layer network would collapse into a single linear function).

Function	Formula	Range	Notes
Step	$f(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{else} \end{cases}$	{0,1}	Non-differentiable at 0; used in original perceptron; not used in modern deep nets (no gradient)
Sigmoid	$f(x) = \frac{1}{1+e^{-(x)}}$	(0,1)	Smooth, differentiable; suffers from vanishing gradient for large x ; used in output layer for binary classification
Tanh	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	(-1,1)	Zero-centered (better than sigmoid for hidden layers); still suffers vanishing gradient at extremes
ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$	[0,∞)	Computationally efficient, mitigates vanishing gradient; can suffer from "dying ReLU" problem (neurons stuck at 0)

7.5 Optimizers

Optimizers determine **how weights are updated** during training to minimize the loss function.

- **Gradient Descent (GD):** Computes gradient using the **entire training dataset** for each update. Accurate but slow/memory-intensive for large datasets.

$$w \leftarrow w - \eta \nabla E(w)$$

- **Stochastic Gradient Descent (SGD):** Updates weights using **one training example at a time** (or a mini-batch). Faster, noisier updates, can escape local minima, but convergence is less stable/smooth.
- **Momentum-based GD:** Adds a fraction of the previous update to the current update, to accelerate convergence and dampen oscillations:

$$\begin{aligned} v &\leftarrow \beta \cdot v + \eta \nabla E(w) \\ w &\leftarrow w - v \end{aligned}$$

Helps navigate ravines (steep in one dimension, flat in another) faster.

- **Adagrad (Adaptive Gradient):** Adapts the learning rate **per-parameter**, based on the historical sum of squared gradients — parameters with large gradients get smaller effective learning rates, and vice versa. Good for sparse data, but learning rate can shrink too much over time (monotonically decreasing).

$$w \leftarrow w - (\eta / \sqrt{G_t + \epsilon}) \cdot \nabla E(w)$$

where G_t = sum of squares of past gradients.

- **Adam (Adaptive Moment Estimation):** Combines the ideas of Momentum (first moment — mean of gradients) and Adagrad/RMSProp (second moment — uncentered variance of gradients). Maintains exponentially decaying averages of past gradients (m_t) and past squared gradients (v_t), with bias correction. Currently one of the most popular optimizers due to fast, stable convergence.

$$\begin{aligned} m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla E(w) \\ v_t &= \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \nabla E(w)^2 \\ w &\leftarrow w - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \end{aligned}$$

8. Support Vector Machines (SVM)

8.1 Motivation and Core Idea

SVM is a **supervised learning** algorithm primarily used for classification. It finds the **optimal separating hyperplane** that best divides the classes — "optimal" meaning it **maximizes the margin** between the two classes.

8.2 Margin and Maximization

- The **margin** is the distance between the separating hyperplane and the nearest data points from either class (the **support vectors**).
- SVM's objective is to find the hyperplane that **maximizes this margin**, since a larger margin generally leads to better generalization on unseen data.
- **Support Vectors** are the data points that lie closest to the decision boundary — they are the only points that influence the position/orientation of the hyperplane (removing non-support-vector points does not change the boundary).

Hyperplane equation: $w \cdot x + b = 0$ **Margin boundaries:** $w \cdot x + b = +1$ and $w \cdot x + b = -1$

Margin width: $2 / ||w||$ — so maximizing margin $\equiv$ minimizing $||w||$ (or $\frac{1}{2} ||w||^2$).

8.3 The Primal Problem

Hard Margin SVM (linearly separable data):

$$\begin{aligned} \text{Minimize:} \quad & (1/2) ||w||^2 \\ \text{Subject to:} \quad & y_i(w \cdot x_i + b) \geq 1, \text{ for all } i = 1 \dots n \end{aligned}$$

This is a **convex quadratic optimization problem** with linear constraints — guarantees a unique global optimum.

Soft Margin SVM (for non-linearly-separable / noisy data): Introduces slack variables $\xi_i \geq 0$ to allow some misclassification/margin violations:

Minimize: $(1/2) ||w||^2 + C \cdot \sum \xi_i$
Subject to: $y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \text{for all } i$

- C is a regularization hyperparameter controlling the tradeoff between maximizing margin and minimizing classification error:
 - **Large C** $\rightarrow$ less tolerance for misclassification $\rightarrow$ narrower margin, may overfit
 - **Small C** $\rightarrow$ more tolerance for misclassification $\rightarrow$ wider margin, may underfit

8.4 The Lagrangian Dual

The primal (constrained) problem is converted into an unconstrained dual problem using **Lagrange multipliers** ($\alpha_i \geq 0$):

Lagrangian (Hard Margin):

$L(w, b, \alpha) = (1/2) ||w||^2 - \sum \alpha_i [y_i(w \cdot x_i + b) - 1]$

Taking partial derivatives w.r.t. w and b and setting to zero gives:

$\partial L / \partial w = 0 \Rightarrow w = \sum \alpha_i y_i x_i$
 $\partial L / \partial b = 0 \Rightarrow \sum \alpha_i y_i = 0$

Substituting back gives the **Dual Problem**:

Maximize: $\sum \alpha_i - (1/2) \sum \sum \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$
Subject to: $\alpha_i \geq 0, \quad \sum \alpha_i y_i = 0$

For Soft Margin, the dual is the same but with an additional upper bound constraint on the multipliers:

$0 \leq \alpha_i \leq C, \quad \sum \alpha_i y_i = 0$

8.5 Solution to the Lagrangian Dual

- Solving the dual gives the optimal α_i values (typically via Quadratic Programming solvers, e.g., SMO — Sequential Minimal Optimization).
- **Support vectors** are exactly those points with $\alpha_i > 0$ — all other points have $\alpha_i = 0$ and don't affect the solution (**sparsity** property).
- The weight vector is recovered as: $w = \sum \alpha_i y_i x_i$ (sum only over support vectors, since other $\alpha_i = 0$).

- Bias (b) is computed using any support vector: $b = y_i - w \cdot x_i$ (for a support vector on the margin).
- **Classification (decision function) for a new point x :**

$$f(x) = \text{sign}(\sum a_i y_i (x_i \cdot x) + b)$$

- Why solve the dual instead of the primal? (1) The dual depends only on **dot products** of input vectors, enabling the **Kernel Trick** for non-linear classification; (2) the dual's constraints are simpler (just box constraints); (3) dual complexity depends on the number of training samples, not the dimensionality of features — useful for high-dimensional data.

8.6 Kernel Functions

When data is not linearly separable in the original feature space, kernels implicitly map data into a **higher-dimensional space** where a linear separator may exist — **without explicitly computing the transformation** (the "kernel trick"), by replacing the dot product $x_i \cdot x_j$ with a kernel function $K(x_i, x_j)$.

- **Linear Kernel:**

$$K(x_i, x_j) = x_i \cdot x_j$$

Used when data is already linearly (or near-linearly) separable — no transformation to higher dimension; equivalent to the standard dot-product SVM.

- **Polynomial Kernel:**

$$K(x_i, x_j) = (x_i \cdot x_j + c)^d$$

where (d) is the degree of the polynomial and (c) is a constant (often ≥ 0). This implicitly maps inputs into a feature space of polynomial combinations of the original features, allowing curved (polynomial) decision boundaries.

Derivation (Linear Kernel dot-product substitution): Starting from the dual objective:

$$\text{Maximize: } \sum a_i - (1/2) \sum \sum a_i a_j y_i y_j (x_i \cdot x_j)$$

Replacing the inner product $(x_i \cdot x_j)$ with a general kernel $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$ (where ϕ is an implicit mapping to higher-dimensional space) gives the **Kernelized Dual**:

$$\text{Maximize: } \sum a_i - (1/2) \sum \sum a_i a_j y_i y_j K(x_i, x_j)$$

For the **linear kernel**, $\phi(x) = x$ (identity mapping), so $K(x_i, x_j) = x_i \cdot x_j$ — this recovers the original (unkernelized) dual exactly, confirming that the linear kernel is simply the special case with no feature transformation.

The decision function generalizes to:

$$f(x) = \text{sign}\left(\sum a_i y_i K(x_i, x) + b\right)$$

Quick Revision Summary Table

Topic	Key Formula / Idea
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$
Precision	$TP/(TP+FP)$
Recall/Sensitivity	$TP/(TP+FN)$
Specificity	$TN/(TN+FP)$
Entropy	$-\sum p(x_i) \log_2(p(x_i))$
Information Gain	$\text{Entropy}(S) - \sum (S_v / S) \text{Entropy}(S_v)$
Bias-Variance	$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$
Perceptron update	$w_i \leftarrow w_i + \eta(t-o)x_i$
Sigmoid	$1/(1+e^{-x})$
ReLU	$\max(0, x)$
SVM margin	$2/\ w\ $
SVM primal (hard)	$\min \frac{1}{2} \ w\ ^2, \text{ s.t. } y_i(w \cdot x_i + b) \geq 1$
SVM dual	$\max \sum \alpha_i - \frac{1}{2} \sum \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$
Polynomial kernel	$(x_i \cdot x_j + c)^d$